Research on the Realizability of Microservice Interaction Contract Based on CSP#

Ruiqiong Wu
School of Software
Yunnan university
Kunming,China
1962327373@qq.com

Qing Duan
School of Software
Yunnan university
Kunming, China
qduan@ynu.edu.cn

Fei Dai
College of Big Data and Intelligent Engineering
Southwest Forestry University
Kunming, China
feidai@ynu.edu.cn

Hongji Yang
Department of Informatics
Leicester University
Leicester, UK
hongji.yang@leicester.ac.uk

Yi Zhang
School of Software
Yunnan university
Kunming,China
407814702@qq.com

Biseng Xie
School of Software
Yunnan university
Kunming, China
530467806@qq.com

***Abstract*—Microservice Architecture is a new development paradigm that transforms the traditional business-oriented information management system into the collaborative work of microservices. Microservices are usually distributed in a loosely coupled manner in the network. In order to coordinate tasks, microservices must coordinate their executions through message interactions with each other. Therefore, modeling and analyzing the interaction between microservices becomes a key issue. The choreography defines the interaction contract between services, and the realizability analysis is the key task to ensure the correct implementation of the microservices interaction contract. The choreography is realizable if interaction contract satisfies choreography specification. This paper use CSP# to analyze the realizability of microservice choreography under synchronous communication and bounded asynchronous communication, and a solution is proposed to repair the unrealizable microservice choreography and made it realizable.**

Keywords*—Model Verification; Microservice; Interaction Contract; CSP#; Realizability

I. INTRODUCTION

With the rapid development of microservice architecture technology, enterprises not only emphasize the reuse of infrastructure, but also the organization of enterprise systems in the form of microservices for effective production and reuse [1]. The introduction of microservices optimizes many problems that traditional frameworks cannot solve, and the composition of microservices increases the flexibility, reusability, and integration of services[9]. The microservice architecture transforms the traditional business-oriented information management system into several microservices, which realizes the loose coupling of the business system, enabling engineers to develop quickly and agilely, system integration and automation operation [3]. In order to coordinate the completion of specific business functions, the key problem to be solved in microservice collaboration is whether microservices can interact correctly and coordinate each other's behavior by message communication. Choreography specifies interaction contracts between services, focusing on message interaction between participants. Realizability is the key task to ensure that the choreography is implemented. Choreography is realizable if the interaction contract between the microservice components satisfies the choreography.

This paper uses CSP# to study the realizability of microservice interaction contract. The main research are: 1) using CSP# to analyze the realizability of microservice choreography under synchronous communication and asynchronous communication; 2) Verify and repair the choreography results. The rest of this paper is organized as follows. Section 2 introduces related works. Section 3 shows our theoretical basis and research approaches. Section 4 presents our realizability analysis experiment. Section 5 proposes a scheme to repair unrealizable choreography. Section 6 makes some concluding remarks.

II. RELATED WORK

There are many works have proposed some technical methods for the realizability analysis of choreography. According to the formal basis, the existing analysis methods of choreography realizability can be roughly divided into three types: the analysis method of choreography realizability based on automata、 process algebra and petri net.

Based on the process algebra LOTOS NT, a coding method of transforming BPMN 2.0 into LOTOS NT is proposed in[10]. Then, LOTOS NT is used to generate the LTS model using CADP tools. Finally, the realizability is verified by behavioral equivalence. For the asynchronous communication model, since

978-1-7281-2607-4/19/$31.00 ©2019 IEEE
DOI 10.1109/COMPSAC.2019.10277

IEEE computer society

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.

LOTOS.NT does not directly support asynchronous communication, which adds a FIFO buffer queue to achieve the content of the message. From the global perspective, [11]uses the automaton as the formal basis, and uses the top-down method to model and analyze the realizability of e-service, and proposes for the first time three sufficient conditions for the realization of session protocol: Lossless Join, Synchronous Compatible and Autonomous; there will also be a situation that a choreography can be realizable, but does not satisfy these three Conditions, so the method also has certain limitations. In[12], the automata is used as the formal basis, which proves that the realizability problem of choreography asynchronous communication is decidable, and the necessary and sufficient conditions for the realizability analysis are proposed. [13]presents a formal framework for the realizability analysis of web services choreography. The biggest contribution of that is the definition of the concept of choreography the layer structure. [2]uses petri net as the formal basis, and proposes: 1) analysis of the realizability of choreography under synchronous communication 2) analysis of the realizability of choreography under bounded asynchronous communication. And the mapping algorithm that maps to the participants is proposed. Finally, the behavioral equivalence is used from the perspective of strong mutual simulation to test the realizability of choreography. The above literature mainly use petri nets, automata and process algebra LOTOS NT as the formal basis.

The above research results have important guiding role and reference for the realizability analysis of this paper. In this paper, CSP# was used for the first time to analyze the realizability of choreography. This paper also conducted experiments to verify our method. By checking the realizability of microservice choreography, a solution is proposed to repair the unrealizable microservice choreography and made it realizable.

III. PROPOSED APPOACH WITH THE MOTIVATING EXAMPLE

This paper uses CSP# to analyze the realizability of the microservice interaction contract, distinguish the realizable service choreography from the unrealizable service choreography, and repair the unrealizable service choreography so that it can be realized. In the paper, CSP# is used to model microservice choreography firstly, then synchronous composition and asynchronous composition of microservice participants are implemented to analyze the realizability of choreography under synchronous and asynchronous communication；and uses behavioral equivalence to verify the behavioral consistency between the original choreography and the choreography realized by participant composition, if the behavior is consistent, the choreography can be realized. If the behavior is inconsistent, the choreography is unrealizable then repair it. It can ensure the effectiveness of interaction and collaboration between microservices, distinguish the realizable microservice choreography and the unrealizable microservice choreography, ensure the correctness of business service modeling and composition reuse. This work can make system components or function reuse more flexible and scalable, overcome the shortcomings of existing technology and methods, and improve the efficiency of system integration within enterprises. To speed up the time of system on-line; improve the efficiency of loosely coupled business services collaborative integration into new business systems; On the functional point of view, through the choreography of micro-services to achieve rapid regeneration of the system, so that the original inherent business functions can be transformed into general business services, which can realize rapid reuse of business logic within enterprises and even between enterprises.

A. Communicating sequential process#(CSP#)

CSP(communicating sequential process) is a formal language for describing the interactions of concurrent and distributed systems[4]. CSP# is an extension of CSP, which integrates high-level modeling operators with low-level procedural codes, for the purpose of efficient mechanical system verification[6].The advantages of using CSP# for analysis are as follows: 1) CSP# can conveniently and effectively model asynchronous communication. 2) CSP# added global variables that can be used to achieve asynchronous communication message buffer. 3) CSP# is supported by the PAT model checker and has been applied to a number of systems, which can model and simulate synchronous communication model and asynchronous communication model..

B. A motivating example

In the section, this paper will show operational semantics of CSP# models, which translates a model into a labeled transition system (LTS). The booking system is used as a running example in the paper. The system contains four participants, namely client(cl), booking system(bs), bank service(bk) and database(db). There are six interaction messages: cl_bs_connect； cl_bs_request； bs_cl_reply； cl_bs_abort；cl_bs_book；cl_bk_pay；bs_db_store. Where " A_ B_message" means that participant A (sender) sends a message to participant B (receiver). For example, cl_bs_connect means that participant cl sends a connection message to participant bs. cl (), bs(), bk() and db() respectively represent the processes corresponding to the four participants. The "→" performs the event and afterwards behaves as another process. Skip = P → Stop, where P is a special event of termination. Stop is the process that does noting.

The choreography process describes the following interaction scenarios:

First, cl sends a connection message (connect) to bs; cl then sends the request message (request1) to bs, and bs sends the response message (reply) to cl. cl then has three options, shown in figure 1 as three branch structures. cl can either send a rejection message to bs (abort), or another request message to bs (request2), or a booking message to bs (book). When cl makes the last choice above, cl sends the payment message (pay) to bk, and finally, bs sends the message (store) storing this transaction to db. In CSP#, these interaction process are declared by CSP# as follows Table 1.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.

Table 1: CSP# of booking system

① cl() = cl_bs_connect ->cl1();

② cl1() = cl_bs_request1 -> bs();

③ bs() = bs_cl_reply -> bs1();

④ bs1() = cl_bs_request2 -> bs() [] cl_bs_abort -> Skip [] cl_bs_book -> bk();

⑤ bk() = cl_bk_pay ->db();

⑥ db() = bs_db_store -> Skip;

Figure 1 is the LTS model by the Table1 CSP# transformed, the sequence of message interaction represented by the model is : cl_bs_connect→bl_bs_request1→bs_cl_reply →(bl_bs_request2)→ cl_bs_book→ cl_bk_pay→bs_db_store ||cl_bs_connect→bl_bs_request1→bs_cl_reply→(bl_bs_request 2)→cl_bs_abort. However, under synchronous and asynchronous communication, a message sequence of cl_bs_connect→bl_bs_request1→bs_cl_reply→ cl_bs_book → bs_db_store→ cl_bk_pay appears in a distributed system of concurrent compositions of participants. In this message sequence, the messages bs_db_store and cl_bk_pay exchange locations. Further analysis shows that in the original arrangement shown in Figure 1, there is a sequence between message bs_db_store and cl_bk_pay. However, in the distributed system built by participants through concurrent combination, the message bs_db_store and cl_bk_pay are concurrency relations. When concurrency is combined, the order between them is uncertain, so the choreography is unrealizable.

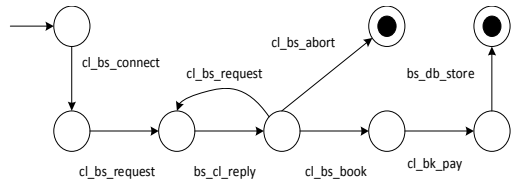


Figure1 The model of booking system's LTS

IV. REALIZABILITY ANALYSIS EXPERIMENT

This paper proposes using behavioral equivalence to analyze the realizability of microservice choreography. More precisely, from the point of view of a weak mutual simulation relationship to detect whether choreography (CSP#) is equivalent to microservice choreography behavior made up of concurrent composite services. If two choreographies are behaviorally equivalent, then the microservice's participant realizes the choreography accurately and the choreography is realizable. If two choreographies do not satisfy behavioral equivalence, the choreography is considered unrealizable. in the next , the research is mainly described with a case study of booking system shows in Figure 1.

A. Realizability analysis of synchronous communication

Synchronous communication is: a sender sends the message, keep waiting for the receiver to send a response message before sending the next message. Essentially, synchronous communication is a kind of block model, generally, synchronous communication requires both sides to establish synchronization before communication, namely to use the same clock frequency, the sender's sending frequency and the receiver's receiving frequency should be synchronized [2]. Table 2 is the synchronous communication composition process of the booking system represented by CSP#. Before CSP# coding, we define seven messages involved in the booking system as global variables. Because the number of messages is 0 before the participants begin to interact, the variables are initialized to 0 when they are defined. This is the Global message sequence definition:

var cl_bs_connect_msg = 0;

var cl_bs_request_msg = 0;

var bs_cl_reply_msg = 0;

var cl_bs_abort_msg = 0;

var cl_bs_book_msg = 0;

var cl_bk_pay_msg = 0;

var bs_db_store_msg = 0;

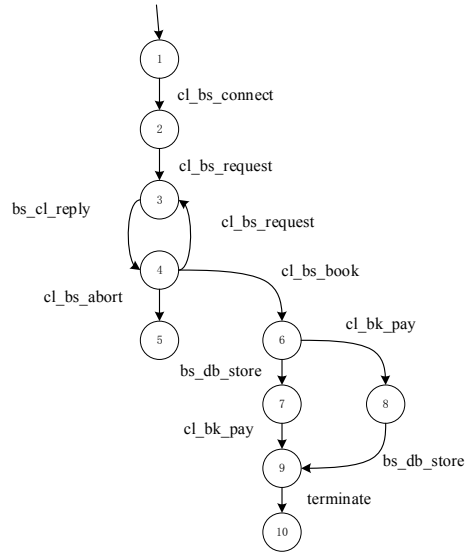


Figure 2 The LTS model of synchronous combination

Table 2 describes that we use CSP# coding the process of synchronous combined communication between participants in

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.

the booking system. In this definition, ①⑥⑪⑫ describes a series of interaction actions in which participants cl, bs, bk and db .Where P1(), P2(), P3() and P4() are the transition process run by P();Q1(), Q2(), Q3() and Q4() are sub-processes of Q(), which describe seven messages of bs interaction. PQRS () represents the concurrent execution process of P (), Q (), R () and S (). Since message senders and receivers use the same clock frequency in synchronous communication, the message buffer size between them is 0, so their message buffer need not be defined in the CSP# encoding of the synchronous communication.

Figure 2 is the LTS model corresponding to CSP# in Table 2.From the point of view of behavior equivalence, it checks whether the LTS shown in Figure 2 and LTS shown in Figure 1 satisfy the weak mutual simulation relation [7]. If the two satisfies the weak mutual simulation relationship, they are equivalent in behavior, then the distributed system accurately realizes the choreography, that is: the choreography is realizable. In the actual experiment, it was founded that participants had one more message bs_db_store-> cl_bk_pay (track 6-> 7-> 9) in the synchronization combination compared with Figure 1. So this is not consistent with the message sequence specified in the original choreography.Therefore, we conclude that synchronous communication is not realizable.

Table 2 CSP# of synchronous composition

① P() = cl_bs_connect -> P1();

② P1() = cl_bs_request -> P2();

③ P2() = bs_cl_reply -> P3();

④ P3() = cl_bs_request -> P2() []
cl_bs_abort -> Skip []cl_bs_book -> P4();

⑤ P4() = cl_bk_pay-> Skip;

⑥ Q() = cl_bs_connect -> Q1();

⑦ Q1() = cl_bs_request -> Q2();

⑧ Q2() = bs_cl_reply -> Q3();

⑨ Q3() = cl_bs_request -> Q2() [] cl_bs_abort -> Skip
[] cl_bs_book -> Q4();

⑩ Q4() = bs_db_store -> Skip;

⑪ R() = cl_bk_pay -> Skip;

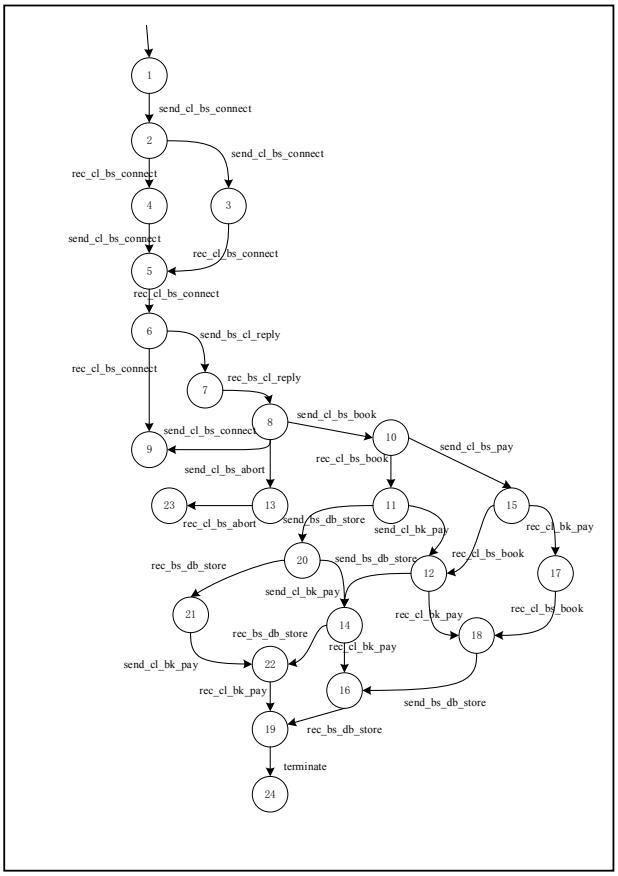
⑫ S() = bs_db_store -> Skip;

⑬ PQRS() = P() || Q() || R() || S();

B. *Realizability analysis of asynchronous communication*

Asynchronous communication: When the sender sends a message, it does not wait for the receiver to send back the response message, and then sends the next message. In essence, asynchronous communication is a non-blocking mode, during the period of message exchange, exchange of messages are stored in receiver's message buffer (message buffer is usually a FIFO queue) until it is consumed and processed. The characteristics of asynchronous communication are as follows: The sender and receiver of a message need not synchronize the sending and receiving operations [2]. Table 2 is the asynchronous communication composition process of the booking system represented by CSP#.

Table 3 describes the interaction process of participants in the booking system under asynchronous communication using CSP#. The four participants correspond to four processes P (), Q (), R () and S (). P() represents a series of actions in which the cl participates in the interaction. P1() indicates that the participant cl sends a request message to bs, and then a message is added to the message buffer of bs, which is cl_bs_request_msg++. P2() shows that bs can receive messages when there is a message in the buffer. At this time, a message in the message buffer is reduced to bs_cl_reply_msg--. Q() represents a series of actions that bs participates in the interaction, and Q1(), Q2(), Q3() and Q4() are a series of sub-processes that bs in the interaction. Similarly, processes R() and S() represent the processes in which participants bk and db interact. ⑬ is PQRS () that denotes the concurrent combination execution process of P (), Q (), R () and



Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.

S ().The difference between asynchronous communication and synchronous communication is that asynchronous communication adds extra message buffer, and the sending message in the P() is shown as: every time a message is sent, the message in the message buffer is added 1;The receiving message in Q() is shown as follows: before each message is received, first verifying whether there is a message in the message buffer. The receiver can start receiving the message when the message in the message buffer is greater than 0. And after receiving is completed, the message in the receiving message buffer is reduced by 1.

Table 3 CSP# of asynchronous composition

```
① P() = send_cl_bs_connect{cl_bs_connect_msg++} -> P1();
② P1() = send_cl_bs_request{cl_bs_request_msg++} -> P2();
③ P2() = [bs_cl_reply_msg > 0]rec_bs_cl_reply
        {bs_cl_reply_msg--} -> P3();
④ P3() = send_cl_bs_request{cl_bs_request_msg++} -> P2()
      [] send_cl_bs_abort{cl_bs_abort_msg++} -> Skip
      [] send_cl_bs_book{cl_bs_book_msg++} -> P4();
⑤ P4() = send_cl_bk_pay{cl_bk_pay_msg++} -> Skip;
⑥ Q() = [cl_bs_connect_msg > 0]rec_cl_bs_connect
        {cl_bs_connect_msg--} -> Q1();
⑦ Q1() = [cl_bs_request_msg > 0]
      rec_cl_bs_request{cl_bs_request_msg--} -> Q2();
⑧ Q2() = send_bs_cl_reply{bs_cl_reply_msg++} -> Q3();
⑨ Q3() = [cl_bs_request_msg > 0]
      rec_cl_bs_request{cl_bs_request_msg--} -> Q2()
        [] [cl_bs_abort_msg > 0]rec_cl_bs_abort
     {cl_bs_abort_msg--} -> Skip [] [cl_bs_book_msg > 0]
        rec_cl_bs_book{cl_bs_book_msg--} -> Q4();
⑩ Q4() = send_bs_db_store{bs_db_store_msg++} -> Skip;
⑪ R() = [cl_bk_pay_msg > 0]rec_cl_bk_pay
        {cl_bk_pay_msg--} -> Skip;
⑫ S() = [bs_db_store_msg > 0]
      rec_bs_db_store{bs_db_store_msg--} -> Skip;
⑬ PQRS() = P() || Q() || R() || S();
```

Figure 3 is the LTS model corresponding to CSP# in table 3. From the perspective of behavioral equivalence, the LTS shown in Figure 3 and Figure 1 are checked whether the LTS satisfies the weak mutual simulation relationship[7].Since asynchronous communication has unbounded messages, it will lead to infinite state space. A method called synchronization analysis is proposed in [5] to solve this problem. If a web service composition is synchronous, we can use synchronous communication instead of asynchronous communication, and its conversation sequence remains unchanged. Therefore, we first verify the synchronization of the booking system, and find that it is synchronized. So we can use synchronous communication instead of asynchronous communication to analyze the realizability of the choreography. At the same time, in figure 3, we also find that participants have an additional session message bs_db_store-> cl_bk_pay in the asynchronous composition (track 11->20->14). Thus this is inconsistent with the message sequence specified in the original choreography. So this paper concludes that the choreography is not realizable in asynchronous communication.

V. WAY TO REPAIR THE CHOREOGRAPHY

The realizability of choreography determines whether the interaction between service participants meets the defined choreography specification. As for the unrealizable choreography, this paper propose a scheme of choreography repair to make it realizable. The significance of the choreography repair is that when an enterprise reuses services, who can first validate whether the composed service is consistent with the prescribed functions. If there is an inconsistency, the service is not realizable. We can repair the unrealizable service to make it reusable perfectly. This also increases the flexibility of service reuse and loose coupling of business systems, enabling engineers to use microservices for rapid and agile development, sustainable system integration, and automated operations and maintenance. In this work, two basic theoretical methods of choreography repaired are introduced: enhancing constraints and relaxing constraints.

- Enhanced constraints: enhanced constraints the behavior $L(C')$ of choreography C' ,makes the behavior L(C) of the original choreography C and the behavior $L(C')$ after choreography repaired is consistent. We assume that the buffer is 1 in the case of asynchronous communication. Moreover, after the choreography is repaired, the behavior $L(I_1^{C'})$ realized by the enhanced constraint is consistent with the behavior $L(I_1^{C})$ implemented by the original choreography, and is the true subset of the original choreography. As shown in formula (1):

$$L(C) = L(C') = L(I_1^{C'}) \subsetneqq L(I_1^{C}) \quad (1)$$

- Relax constraints: after relaxing constraints the behavior $L(C')$ of choreography of C' is richer than the behavior L(C) of the original choreography C. That is, the behavioral interaction sequence L(C) of the original choreography is the subset of repaired behavior interaction sequence $L(C')$. Besides, the behavior $L(I_1^{C'})$ of the repaired choreography implemented is consistent with the behavior $L(I_1^{C})$ by the original choreography. We assume that

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.

the buffer is 1 in an asynchronous communication environment. Formula (2) is shown as follows:

$$L(C) \subsetneqq L(C') = L(I_1^{C'}) \subsetneqq L(I_1^{C}) \quad (2)$$

VI. CONCLUSION AND FUTURE DIRECTIONS

This paper proposes to use CSP# to analyze the microservice interaction contracts. This is the first time that CSP# is used as a formal basis to research the realizability of microservice interaction contracts. This paper also propose a repair scheme for the unrealizable choreography. This paper intends to support a popular enterprise development mode "microservice", which provides good flexibility and extensibility for the reuse of enterprise system components and functions, enables engineer to develop quickly and agile, and greatly saves the development cost of enterprises. In the future, We will continue our research and integrate a visual tool. The tool will support automatically verifying the realizability of micro-service choreography, and automatically repairing the non-realizable micro-service choreography.

ACKNOWLEDGMENT

This work has been partially supported by the Open Foundation of Key Laboratory in Software Engineering of Yunnan Province, China under Grant No. 2017SE101.

.

REFERENCES

[1] Jacobson I. Software Reuse Architecture, Process and organization for business Success[M]. USA: Pearson Education Limited,1993:3-6

[2] Fei Dai, Yabo Wang, Bi Huang, Ruiqiong Wu, Yong Cao,. Analyzing Realizability of Choreographies using Petri Nets. Journal of software, 2018

[3] Fu X, Bultan T, Su J. Synchronizability of Conversations among Web Services[J]. IEEE Transactions on Software Engineering, 2006, 31(12):1042-1055.

[4] Jun Sun, Yang Liu, Jin Song Dong and Chunqing Chen. Integrating Specification and Programs for System Modeling and Verification. Third IEEE International Symposium on Theoretical Aspects of Software Engineering, 29-31 July 2009, Tianjin, China.

[5] Fu X, Bultan T, Su J. Synchronizability of Conversations among Web Services[J]. IEEE Transactions on Software Engineering, 2006, 31(12):1042-1055..

[6] Lukas Ladenberger. Ivaylo Dobrikov. Michael Leuschel. An Approach for Creating Domain Specific Visualisations of CSP Models. Conference Paper · September 2014.

[7] Milner R. Communication and Concurrency[M]. Prentice Hall, 1989

[8] Lewis J，Fowler M. Microservices［EB/OL］.［2016 －06 －25］ https: // martinfowler. com / articles / microservices.html.

[9] Catalin S，Ocatavian D，R oxana S，et al. Software architectures-present and visions［J］. Informatica Economica，2015，19 (4) : 13 － 26.

[10] Zaha JM, Barros A, Dumas M, Hofstede AT. Let's Dance: A Language for Service Behavior Modeling[C]. Proceedings of the 14th International Conference on Cooperative Information Systems, 2006, 145-162.

[11] Decker G, Weske M. Local Enforceability in Interaction Petri Nets[M]. Business Process Management. Springer Berlin Heidelberg, 2007:305-319.

[12] Aalst W M P V D, Basten T. Inheritance of workflows: an approach to tackling problems related to change[J]. Theoretical Computer Science, 2002, 270(1):125-203.

[13] Dingle NJ, Knottenbelt WJ and Suto T. PIPE2: A Tool for the Performance Evaluation of Generalised Stochastic Petri Nets. ACM SIGMETRICS Performance Evaluation Review ,2009,36(4): 34-39.

[14] Pascal Poizat, Gwen Salaün. Checking the Realizability of BPMN 2.0 Choreographies. 27th Sympo-sium On Applied Computing (SAC 2012), Mar 2012, Italy. pp.1927-1934, 2012.

[15] Fu X, Bultan T, Su J. Conversation Protocols: A Formalism for Specification and Verification of Reactive Electronic Services[J]. Theoretical Computer Science, 2004, 328(1):19-37.

[16] R. Kazhamiakin and M. Pistore. Analysis of Realizability Conditions for Web Service Choreographies. In Proc. of FORTE'06, volume 4229 of LNCS, pages 6176. Springer, 2006.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:10:27 UTC from IEEE Xplore. Restrictions apply.